Streams: Laziness and the Pipeline

Why it matters

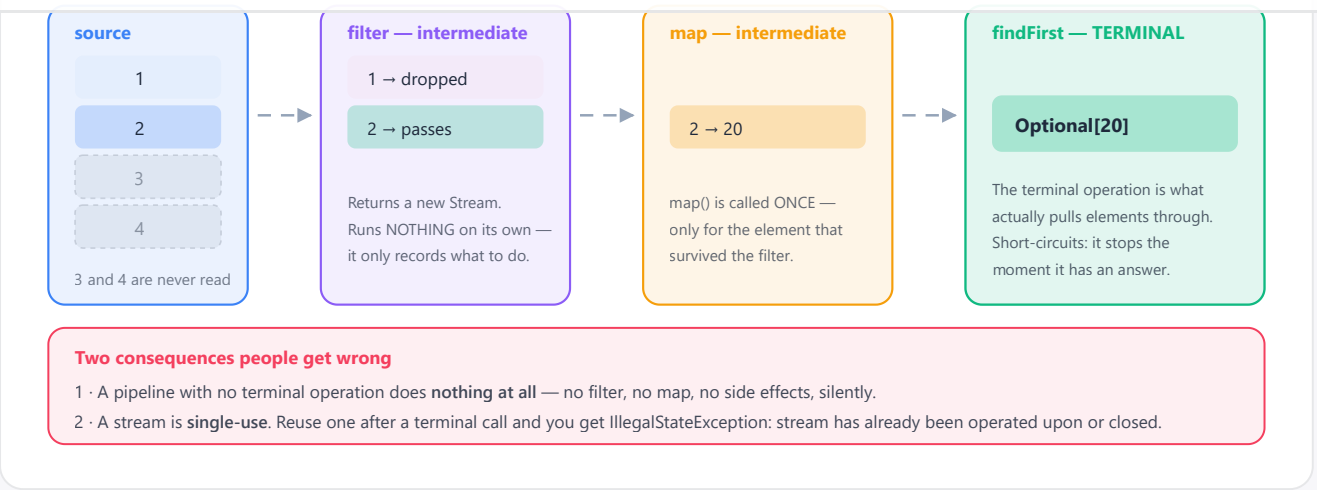
Streams are the most-used Java 8 feature and the most misunderstood. The mental model most people carry — "it's a loop with nicer syntax" — predicts the wrong answer for every interesting question: why nothing happens without a terminal operation, why `findFirst` on a million elements is instant, why a stream cannot be reused. All three follow from one property: **streams are lazy**.

The concept, simply

A stream is not a collection. It holds no data — it is a *description of a computation* over a source. Nothing executes until a **terminal operation** asks for a result.

Operations come in two kinds:

- **Intermediate** — `filter`, `map`, `sorted`, `distinct`, `limit`, `peek`. Return a new stream, and do *nothing* on their own.
- **Terminal** — `collect`, `forEach`, `reduce`, `count`, `findFirst`, `anyMatch`, `toList`. Trigger the whole pipeline and produce a result.



Elements flow one at a time through the entire pipeline — not stage by stage over the whole collection. That is why findFirst never touched elements 3 and 4.

How it works

The processing model is **vertical, not horizontal**. A loop-based mental model says "filter everything, then map everything". A stream instead pulls one element all the way through, then the next. That single difference explains the useful behaviour:

- **Short-circuiting.** `findFirst`, `anyMatch`, `limit` stop as soon as they can answer, so an expensive `map` runs only on the elements actually needed.
- **No intermediate collections.** A chain of five operations allocates no five temporary lists.
- **Infinite streams work.** `Stream.iterate` with a `limit` terminates, because only the requested elements are ever produced.

A stream is single-use. After a terminal operation it is consumed; touching it again throws `IllegalStateException: stream has already been operated upon or closed`. If you need two results, either collect once and reuse the collection, or build the stream twice from the source.

Stateful operations break the flow. `sorted` and `distinct` cannot emit anything until they have seen enough input — `sorted` must consume the entire stream before producing its first element. Put them as late in the pipeline as possible, after the filters that shrink the data.

Laziness, made visible:

JAVA

Copy

```
List<String> words = List.of("apple", "banana", "cherry", "date");

Optional<String> result = words.stream()
    .peek(w -> System.out.println("filter sees: " + w))
    .filter(w -> w.length() > 5)
    .peek(w -> System.out.println("  map sees: " + w))
    .map(String::toUpperCase)
    .findFirst();

// filter sees: apple
// filter sees: banana
//   map sees: banana
// → BANANA
//
// "cherry" and "date" are never read at all, and map ran exactly once.
// A loop-shaped mental model would predict four filters and three maps.
```

The silent no-op — the bug this topic exists to prevent:

JAVA

Copy

```
// ✗ Does absolutely nothing. No exception, no warning, no output.
list.stream().map(this::save);          // no terminal operation

// ✓ Terminal operation present.
list.stream().map(this::save).toList();
list.forEach(this::save);               // simpler still when you just wan

// ✗ Single-use:
Stream<String> s = list.stream();
s.count();
// s.forEach(System.out::println);      → IllegalStateException

// ✓ Build it twice, or collect once:
List<String> filtered = list.stream().filter(this::keep).toList();
```

Ordering matters for performance, and sometimes for correctness:

JAVA Copy

```java
// ✗ Sorts 1,000,000 elements, then throws almost all of them away.
list.stream().sorted().filter(this::isActive).limit(10).toList();

// ✓ Filter first — sorted() now handles far fewer elements.
list.stream().filter(this::isActive).sorted().limit(10).toList();

// ⚠ sorted() is STATEFUL: it must consume the entire stream before emitti
//   anything, so it defeats short-circuiting and never terminates on an
//   infinite stream. Same for distinct(), which must remember what it has
```

Creating streams, including the ones people forget:

JAVA Copy

```java
Stream.of("a", "b", "c");
Arrays.stream(array);
list.stream();
IntStream.range(0, 10);                    // primitive — no boxing
IntStream.rangeClosed(1, 10);
Stream.iterate(1, n -> n * 2).limit(10);   // infinite + limit
Stream.generate(Math::random).limit(5);
"a,b,c".chars();                            // IntStream of code points
Pattern.compile(",").splitAsStream("a,b,c");

// ⚠ Holds an open file handle — must be closed (Chapter 6):
try (Stream<String> lines = Files.lines(path)) {
    lines.filter(l -> !l.isBlank()).forEach(this::handle);
}
```

Under the hood

Internally a stream is a linked chain of stages, each wrapping a `Sink`. The terminal operation drives a `Spliterator` over the source and pushes each element into the head of the chain; each stage does its work and pushes downstream. Short-circuiting is implemented by a stage signalling that it has had enough, which propagates back up and stops the traversal.

`peek` **is for debugging only**, and there is a subtle reason beyond style. Because it is an intermediate operation, it only runs on elements the pipeline actually pulls — so a `peek` before a short-circuiting terminal will appear to "miss" elements. Worse, the JDK is explicitly allowed to elide it: in some cases `count()` can skip the entire pipeline if it can compute the size from the source, and your `peek` never runs at all. Never put logic in a `peek`.

Primitive streams exist for a real reason. `Stream<Integer>` boxes at every stage; `IntStream` does not. On large numeric pipelines the difference is several times, not a few percent. Convert with `mapToInt` / `mapToObj`, and remember `IntStream` has `sum()`, `average()` and `summaryStatistics()` that the object stream lacks.

Streams versus loops: the honest position is that a stream is usually clearer for a transformation pipeline and a loop is usually clearer — and marginally faster — for simple iteration with side effects. Streams have real startup overhead per pipeline, so for a three-element list in a hot loop, a `for` loop wins. Claiming streams are always better is a weaker answer than knowing when they are not.

Common mistakes

- **Forgetting the terminal operation.** The pipeline silently does nothing.
- **Reusing a stream.** Single-use — collect once instead.
- **Putting `sorted` before `filter`.** Sorts data you are about to discard.
- **Logic inside `peek`.** It may not run at all. Debugging only.
- **Mutating shared state from a lambda.** Breaks under parallelism and hides intent. Use a collector.
- **Boxing in numeric pipelines.** Use `IntStream` / `LongStream` / `DoubleStream`.
- **Not closing `Files.lines()`.** Leaks a file handle.

- **Modifying the source during a stream.** Undefined behaviour — sometimes silent, sometimes `ConcurrentModificationException`.

Interview lens

Q: "What is the difference between an intermediate and a terminal operation?"

A: Intermediate operations return a new stream and are lazy — they record what to do and execute nothing. Terminal operations produce a result and are what actually drive the pipeline. A chain with no terminal operation does nothing at all, silently, which is one of the most common stream bugs.

Q: "How does laziness actually help?"

A: Three ways. Short-circuiting — `findFirst` or `anyMatch` stops as soon as it has an answer, so expensive mappings run only on the elements needed. No intermediate collections, because elements flow one at a time through the whole chain rather than stage by stage. And infinite streams become usable, since `Stream.iterate( ... ).limit(n)` only ever produces n elements.

Q: "Can you reuse a stream?"

A: No. Once a terminal operation runs, the stream is consumed and any further use throws `IllegalStateException`. If you need several results, collect to a list once and work from that, or create a fresh stream from the source each time.

Q: "Does operation order matter?"

A: A lot. Filtering before mapping or sorting means the expensive stages see fewer elements. `sorted` and `distinct` are stateful — `sorted` must consume the entire stream before it can emit anything — so they defeat short-circuiting and never terminate on an infinite stream. Put them as late as possible.

A: Debugging, and nothing else. It is intermediate, so it only sees elements the pipeline actually pulls, and the JDK is explicitly permitted to skip it — `count()` can bypass the pipeline entirely when it can get the size from the source, and your `peek` never runs. Any logic that must happen belongs in a `map` or the terminal operation.

Q: "Are streams faster than loops?"
A: Usually not, for simple iteration — there is per-pipeline setup cost, so a plain `for` loop wins on small collections. What streams buy is clarity for transformation pipelines and the option of parallelism. For numeric work the primitive streams matter more than the stream-versus-loop question, because `Stream<Integer>` boxes at every stage.

Cheat-card

- Streams hold **no data** — they describe a computation. **Lazy** until a terminal op.
- Intermediate (`filter`, `map`, `sorted`) return a stream; terminal (`collect`, `count`, `findFirst`) run it.
- **No terminal = nothing happens**, silently.
- Elements flow **one at a time through the whole chain** — hence short-circuiting.
- **Single-use** — reuse throws `IllegalStateException`.
- `sorted` / `distinct` are **stateful**: put them last, never on an infinite stream.
- `peek` = **debugging only**; it may be elided entirely.
- Numeric work → `IntStream` / `LongStream` / `DoubleStream`, no boxing.
- `Files.lines()` must be **closed**.

QUICK CHECK — no marks, just for you.

1. What does list.stream().map(this::save) do on its own?

- Saves every element
- Nothing at all — with no terminal operation the pipeline never executes, silently and without warning
- Throws IllegalStateException

Check answer

2. In stream().filter(f).map(m).findFirst() over four elements where the second matches, how many times does m run?

- Four times, once per element
- Three times
- Twice
- Once — elements flow one at a time through the whole pipeline and findFirst short-circuits, so elements 3 and 4 are never read

Check answer

3. Why should peek() never contain logic?

- Because it only sees elements the pipeline actually pulls, and the JDK is explicitly permitted to elide it — count() can skip the pipeline entirely when it can get the size from the source
- Because it cannot access the element
- Because it runs on a different thread
- Because it is a terminal operation

Previous
Lambdas and Functional Interfaces

Next
Collectors, groupingBy and Reduction

LEARN MORE

ARTICLES & DOCS

- java.util.stream — package overview
  docs.oracle.com
- Stream — API docs
  docs.oracle.com

VIDEOS



Streams API Tutorial in Java 8+
Daily Code Buffer



Functional Programming with Java Streams API
Amigoscode

Hand-picked references for this lesson.